Le classi

Introduzione alle classi in Java

Nelle lezioni precedenti abbiamo introdotto il **paradigma orientato agli oggetti**, chiarendo la differenza tra **classe** e **oggetto**, il ruolo dei **costruttori**, delle **reference**, dei **package** e dei **modificatori di visibilità**.
Abbiamo visto come una classe rappresenti un **tipo astratto**, mentre gli oggetti siano le **istanze concrete** create a runtime, e come Java affidi al **runtime** la gestione della memoria tramite il Garbage Collector.

Abbiamo inoltre analizzato:

- la struttura di una classe dal punto di vista concettuale (attributi e metodi)
- l'importanza dell'incapsulamento
- l'uso di tipi avanzati come le `enum`, che permettono di definire categorie sicure e riusabili

In questa nuova parte della lezione ci concentriamo sulla **struttura sintattica di una classe Java**, su come viene definita nel codice e su alcune **regole pratiche di organizzazione dei file**, fondamentali per scrivere programmi corretti e ben strutturati.

Definizione di una classe

Una classe in Java può essere definita secondo la seguente forma generale:

Java

```java
public class NomeClasse
{
    // dichiarazione degli attributi
    // definizione dei metodi
}
```

All'interno della classe troviamo:

- la **dichiarazione degli attributi** → **che rappresentano lo stato degli oggetti**
- la **definizione dei metodi** → **che ne rappresentano il comportamento**

È possibile scrivere **due o più classi nello stesso file**, ma vale una regola fondamentale:

- **solo una classe può essere `public`**
- **il nome del file deve coincidere con il nome della classe pubblica**

Le altre classi definite nello stesso file:

- **non sono** `public`
- **sono visibili solo all'interno del package**

> **Nota sull'IDE (Eclipse)**
>
> In Eclipse, quando si crea una nuova classe, nella finestra di creazione è presente:
>
> - **il campo Name per il nome della classe**
> - la possibilità di scegliere la visibilità:
> - `public`
> - `package` (visibilità di default)
>
> Questa scelta influisce direttamente sulla **accessibilità della classe** all'interno o all'esterno del package.

Definizione di pacchetto

Come già spiegato nella scorsa lezione, un **pacchetto (package)** è:

- **una directory che ha lo scopo di organizzare i file** `.class` **che compongono una libreria o un'applicazione Java.**

Dal punto di vista del runtime, i veri elementi organizzati nei package sono i **file** `.class`,

- **che vengono disposti in cartelle gemelle della struttura dei package dichiarati nel codice sorgente.**

Il concetto di package ha **due aspetti distinti ma complementari**.

1. Organizzazione logica dei nomi

- **ogni package rappresenta uno spazio di nomi (namespace) distinto**

- **classi con lo stesso nome possono esistere in package diversi senza conflitti**

2. Organizzazione fisica dei file

- **ogni package corrisponde a una cartella distinta sul disco**
- **la gerarchia dei package viene riflessa nella struttura delle directory**

Ad esempio, il package della libreria standard:

```java
java.util
```

è in realtà una **gerarchia di sotto-package**, dove `util` è un sotto-package di `java`.

Nome di un pacchetto

Il package di una classe viene dichiarato nel file `.java` tramite l'istruzione:

```java
package myPackage;
```

Questa istruzione:

- **deve essere la prima istruzione del file**
- **associa la classe a uno specifico package**
- **determina anche il percorso fisico in cui verrà collocato il file** `.class`

Il nome di un package deve essere il più possibile univoco.
Per questo si utilizza una convenzione basata sul **dominio inverso**, ad esempio:

```plain text
it.nomePacchetto.nomeModulo
```

Questa convenzione riduce il rischio di collisioni tra package e favorisce la riusabilità del codice.

Uso dei pacchetti

Per utilizzare una classe che appartiene a un altro package è necessario usare l'istruzione `import`.

```java
import myPackage.MyClass;
import java.util.Date
```

Per utilizzare una classe che appartiene a un altro package è necessario usare l'istruzione `import`.

Il default package

Se in Eclipse si crea una classe **senza specificare un package**, l'IDE la colloca in un cosiddetto `default package`.

È importante notare che:

- **il *default package* esiste solo a livello logico**
- **nel file system non viene creata alcuna directory**
- **di conseguenza, le classi nel default package non possono essere importate da altre classi che si trovano in package espliciti**

Questo accade perché Java richiede un **percorso completo** (ad esempio `java.util.Date`) **per poter importare una classe, e tale percorso è definito solo se la classe appartiene a un package reale.**

Per questo motivo, l'uso del default package è accettabile solo per:

- **esempi semplici**
- **esercizi didattici**

> ⚡ **Di conseguenza il defualt package fortemente sconsigliato nei progetti reali.**

Regole di Java sui pacchetti

Java impone alcune regole fondamentali:

- **tutte le classi devono appartenere a un package**
- **se il programmatore non ne specifica uno, il compilatore assegna automaticamente la classe al default package**

- **le classi nel default package non sono importabili**

In pratica, per scrivere codice:

- **modulare**
- **riusabile**
- **correttamente organizzato**

è sempre buona norma **definire esplicitamente un package** per ogni classe.

Attributi di una classe

Come già accennato nella lezione precedente, gli **attributi** (detti anche *campi* o *variabili di istanza*) rappresentano:

- **lo stato interno di un oggetto e vengono dichiarati all'interno della classe.**

La sintassi generale è la seguente:

Java

```java
tipo1 nomeVariabile1;
tipo2 nomeVariabile2;
```

Tipi degli attributi

Il tipo di un attributo può essere:

- **tipo primitivo (`int`, `double`, `boolean`, ecc.)**
- **classe di libreria (ad esempio `String`, `Date`, `ArrayList`)**
- **classe definita dal programmatore**

Per utilizzare classi che non appartengono allo stesso package è necessario importarle:

Java

```java
import package1.Class1
import package2.*;
```

L'istruzione `import` non carica fisicamente la classe, ma consente al compilatore di:

- **riconoscerne il nome senza dover usare il percorso completo.**

Accesso agli attributi

Gli attributi di un oggetto possono essere utilizzati tramite la **notazione punto (dot notation)**:

Java

```java
nomeOggetto.attributo;
```

Tuttavia, **la possibilità di accedere direttamente a un attributo dipende da due fattori fondamentali**:

1. **Da dove avviene l'accesso**
   (stessa classe, stesso package, sottoclasse, classe esterna)
2. **Dalla visibilità dell'attributo**
   (`public` , `protected` , default, `private`)

> ⚠ **Dal punto di vista sintattico, l'accesso diretto agli attributi è consentito da Java, ma dal punto di vista progettuale è spesso sconsigliato.**

Incapsulamento

L'**incapsulamento** è uno dei **principi fondamentali della programmazione a oggetti** e consiste nel:

- **nascondere i dettagli interni di una classe, esponendo verso l'esterno solo ciò che è realmente necessario.**

In pratica, una classe:

- **incapsula il proprio stato (gli attributi).**
- **controlla il modo in cui questo stato può essere letto o modificato**.

Per rispettare il principio di **incapsulamento**:

- **gli attributi vengono in genere dichiarati** `private`
- **l'accesso avviene tramite metodi pubblici (**getter **e** setter**)**

Perché usare l'incapsulamento

L'incapsulamento permette di:

- **controllare l'uso degli attributi, ad esempio validando i valori prima di assegnarli**
- **proteggere lo stato interno dell'oggetto, evitando modifiche non previste**
- **ridurre l'accoppiamento tra le classi**
- **rendere il codice più manutenibile e robusto, perché le modifiche interne alla classe non impattano il codice che la utilizza**

Questo significa che **il codice esterno non può accedere direttamente agli attributi**, ma deve passare attraverso metodi che la classe mette a disposizione. Quindi per rispettare il principio di **incapsulamento**:

- **gli attributi vengono in genere dichiarati** `private`
- **l'accesso avviene tramite metodi pubblici (getter e setter)**

Questo permette di:

- controllare l'uso degli attributi
- proteggere lo stato interno dell'oggetto
- rendere il codice più manutenibile e robusto

Metodi di una classe

I metodi "normali" di una classe sono detti **metodi di istanza**:

- **essi operano su una specifica istanza (oggetto) della classe e possono accedere agli attributi dell'oggetto stesso.**
  Questo concetto è analogo ai metodi delle classi in Python.

I metodi di istanza:

- **possono ricevere da zero a n parametri**
- **possono restituire un valore oppure non restituire nulla**
  - nel primo caso è obbligatorio dichiarare il **tipo di ritorno**
  - nel secondo caso si utilizza il tipo `void`

Struttura generale di un metodo

Un metodo di istanza ha la seguente forma generale:

```java
public/private tipoRitornato nomeMetodo(elenco parametriRicevuti)
{
    dichiarazioni di variabili locali;
    istruzioni;
    ..................;
    return valore; // opzionale, solo se tipoRitornato ≠ void
}
```

Dove:

- `public` / `private` → **definiscono la visibilità del metodo**
- `tipoRitornato` → **indica il tipo del valore restituito (oppure `void`)**
- `nomeMetodo` → **è l'identificatore del metodo**
- `elenco parametriRicevuti` → **è la lista dei parametri formali**
- il corpo del metodo contiene le istruzioni da eseguire

Invocazione dei metodi

I metodi di istanza vengono invocati tramite la **notazione punto** sull'oggetto:

```java
nomeOggetto.metodo(parametri);
```

Il metodo viene eseguito **sull'istanza specificata**, utilizzando i valori degli attributi di quell'oggetto.

Costruttori e modificatori

In Java, il **costruttore** è:

- **un metodo speciale che serve a creare e inizializzare un oggetto della classe.**

Caratteristiche principali:

1. **Il nome del costruttore è sempre uguale al nome della classe.**
2. Non si dichiara alcun tipo di ritorno:
   - **il "valore restituito" implicito è l'oggetto in costruzione.**
3. **Il costruttore ha il compito di valorizzare tutti gli attributi della classe al momento della creazione dell'oggetto.**

Esempio: classe `Impiegato`

Java

```java
public class Impiegato {
    private String nome;
    private double salario;
    private Date dataAss;

    public Impiegato(String n, double s, Date d) {
        nome = n;
        salario = s;
        dataAss = d;
    }
}
```

In questo esempio:

- **il costruttore prende tre parametri (`n` , `s` , `d`) e li assegna agli attributi dell'oggetto**
- **il costruttore inizializza gli attributi, così l'oggetto è pronto per essere utilizzato**

Comportamento degli attributi non inizializzati

Se si dimentica di valorizzare un attributo nel costruttore:

- Java assegna automaticamente **un valore di default**, proprio come avviene per gli array.
  - Numeri (`int` , `double` , `float`) → 0
  - Boolean → false
  - Riferimenti a oggetti → null

In questo esempio, il `this` non è necessario perché i nomi dei parametri (`n` , `s` , `d`) **sono diversi dai nomi degli attributi** (`nome` , `salario` , `dataAss`).
Se i nomi coincidessero, allora sarebbe necessario usare `this` per distinguere l'attributo della classe dal parametro del costruttore (questo comportamento è detto shadowing):

Java

```java
public Impiegato(String nome, double salario, Date dataAss) {
    this.nome = nome;
```

```
        this.salario = salario;
        this.dataAss = dataAss;
    }
```

> 🧠 **Il costruttore è quindi il punto di partenza per qualsiasi oggetto:**
>
> - **definisce il suo stato iniziale e permette di creare istanze coerenti con le regole della classe.**

La keyword `this`

La keyword `this` in Java serve a:

- **riferirsi all'oggetto che sta chiamando il metodo o il costruttore.**

Un caso tipico in cui `this` diventa necessario è lo **shadowing:**

- **cioè quando i parametri di un metodo o di un costruttore hanno lo stesso nome degli attributi della classe.**
  In questo caso, `this` distingue l'attributo dell'oggetto dalla variabile locale.

Java

```java
public class Impiegato {
    private String nome;

    public Impiegato(String nome) {
        this.nome = nome;  // l'attributo nome dell'oggetto viene valorizzato con il parametro
    }
}
```

Senza `this`, il compilatore **considererebbe solo la variabile locale `nome` e non riuscirebbe ad aggiornare l'attributo dell'oggetto.**

Valori di default degli attributi

Se un attributo non viene valorizzato esplicitamente nel costruttore:

- **Oggetti** (comprese le `String`) → `null`
- **boolean** → `false`

- **char** → `\u0000` (carattere nullo)
- **Numeri** (`int` , `double` , `float` , ecc.) → `0`

📋 Confronto: `this` in Java, `this` in JavaScript e `self` in Python

In Java, come visto, la keyword `this` :

- **si riferisce all'oggetto corrente su cui il metodo o il costruttore è stato invocato.**
  Serve a distinguere gli attributi dell'oggetto dalle variabili locali quando hanno lo stesso nome (shadowing).

In altri linguaggi orientati agli oggetti ci sono concetti simili, ma con alcune differenze:

JavaScript

- Anche in JavaScript esiste `this` , ma **il suo significato dipende dal contesto di esecuzione.**
  - In un **metodo di oggetto**, `this` **si riferisce all'oggetto stesso** .
  - In una **funzione globale** o in una funzione normale chiamata senza oggetto, `this` **può riferirsi all'oggetto globale (** `window` **nel browser) o essere** `undefined` **in modalità strict** .
- In sintesi: **in JavaScript** `this` **non è sempre l'oggetto che stai istanziando, il suo valore dipende da come viene chiamata la funzione** .

Python

- In Python il concetto equivalente è `self` , che però **è un parametro >esplicito del metodo**:

Python

```python
class Impiegato:
  def __init__(self, nome):
      self.nome = nome
```

- `self` rappresenta sempre **l'oggetto corrente**, ma **deve essere dichiarato come primo parametro** dei metodi di istanza.

- Non esiste uno shadowing implicito come in Java: ogni riferimento a un attributo dell'oggetto deve passare tramite `self`.

Differenze principali tra `self` e `this`

Concetto	Java (`this`)	Python (`self`)	JavaScript (`this`)
Dichiarazione	Implicita, non si passa come parametro	Esplicita, primo parametro dei metodi	Implicita, dipende dal contesto di invocazione
Shadowing	Serve per distinguere variabili locali da attributi	Sempre necessario usare `self`	Dipende dal contesto; può cambiare dinamicamente
Riferimento all'oggetto	Sempre l'oggetto corrente	Sempre l'oggetto corrente	Può variare: oggetto chiamante, globale, undefined

In breve:

- **`this` in Java è più prevedibile di quello in JavaScript,**
- **mentre `self` in Python è completamente esplicito, rendendo chiaro in ogni metodo quale oggetto si sta modificando.**

Il costruttore di default

Se **non viene scritto nessun costruttore**, il compilatore Java crea automaticamente un **costruttore di default**:

- **senza parametri**
- **con tutti gli attributi inizializzati ai valori di default (salvo eventuali assegnazioni esplicite nella dichiarazione)**

Questo costruttore esiste **solo nel bytecode del** `.class`:

- **il programmatore non lo vede nel file sorgente, ma Java lo usa internamente per permettere la creazione di oggetti.**

Per questo motivo viene talvolta chiamato **costruttore di emergenza** o **costruttore di disperazione**.

Overloading dei metodi

In Java è possibile definire più metodi con lo stesso nome, purché la lista dei parametri sia diversa.
Questa caratteristica si chiama **overloading dei metodi**.

L'overloading permette di creare metodi "simili" che svolgono la stessa funzione **ma accettano tipi o numeri diversi di argomenti, rendendo il codice più leggibile e flessibile**.

Esempio di metodi validi in overloading:

Java

```java
String metodo(int i, Date d) { ... }
String metodo(Date d) { ... }
String metodo(Date d, int i) { ... }
```

Tutti e tre i metodi hanno lo stesso nome, ma Java li distingue grazie **alla diversa sequenza o tipo dei parametri**.

⚠ **Attenzione:**

- Nell'overloading **il tipo di ritorno non conta**.
- Non basta cambiare solo il tipo di ritorno, ad esempio:

```java
double metodo(Date d, int i) { ... }
```

non compila se esiste già il metodo `String metodo(Date d, int i)` perché la firma del metodo (nome + lista parametri) deve essere unica.

In sintesi, **la firma del metodo in Java è definita dal nome e dalla lista dei parametri**, non dal tipo restituito.

Overriding dei metodi

L'**overriding,** come in Python, è un concetto legato all'**ereditarietà:**

- **permette a una sottoclasse di fornire una nuova implementazione di un metodo già definito nella superclasse.**

In altre parole:

- **Il metodo nella superclasse definisce il comportamento "base".**
- **La sottoclasse può ridefinirlo per adattarlo alle proprie esigenze.**

Regole principali dell'overriding in Java

1. **Il nome del metodo e la lista dei parametri devono essere identici a quelli del metodo della superclasse**.
2. **Il tipo di ritorno deve essere compatibile** (può essere lo stesso o un sottotipo, con il cosiddetto *covariant return type*).
3. **Il metodo della superclasse non può essere `final` (perché i metodi finali non si possono sovrascrivere)**.
4. **La visibilità della sottoclasse deve essere uguale o più permissiva di quella del metodo della superclasse.**
   Esempio:

Java

```java
class Animale {
    public void verso() {
        System.out.println("L'animale fa un verso");
    }
}

class Cane extends Animale {
    @Override
    public void verso() {
        System.out.println("Il cane abbaia");
    }
}
```

In questo caso:

- `Cane` **ridefinisce** il metodo `verso()` della superclasse `Animale`.
- Quando chiamiamo `verso()` su un oggetto `Cane`, viene eseguito **il metodo della sottoclasse**, non quello della superclasse.

Differenza tra Overloading e Overriding

In Java esistono due modi principali per avere più metodi con lo stesso nome, ma servono a cose diverse: **overloading** e **overriding**.

1. **Overloading**
   L'overloading significa che:

- **nella stessa classe possiamo avere più metodi con lo stesso nome, purché cambino i parametri (numero o tipo)**.
- Serve a rendere più leggibile il codice quando vogliamo fare la stessa operazione con input diversi.
- Ad esempio, una calcolatrice può avere un metodo `somma(int a, int b)` per interi e un altro `somma(double a, double b)` per numeri decimali.
- In questo caso Java decide quale metodo chiamare in base ai **parametri effettivamente passati**.
- **Il tipo di ritorno può cambiare liberamente e non conta per distinguere i metodi**.

2. **Overriding**
   L'overriding, invece, riguarda le **classi in gerarchia**:

- **una sottoclasse può ridefinire un metodo della superclasse con lo stesso nome e la stessa lista di parametri**.
- **Lo scopo è modificare o specializzare il comportamento di un metodo ereditato.**
- Ad esempio, la classe `Animale` può avere un metodo `verso()`. La sottoclasse `Cane` lo ridefinisce con `verso()` per far sì che il cane abbaia invece di fare un verso generico.
- Il tipo di ritorno deve essere compatibile con quello della superclasse, e il metodo della superclasse non deve essere `final`.
- Quando chiamiamo il metodo su un oggetto della sottoclasse, viene eseguito **il metodo ridefinito**, anche se la variabile è di tipo della superclasse.

> **In breve:**
> - **Overloading** = **stessa classe, stesso nome, parametri diversi** → scegliamo quale usare in base agli argomenti.

- **Overriding** = **sottoclasse ridefinisce metodo della superclasse** → scegliamo quale usare in base all'oggetto reale.

Quindi, overloading riguarda:

- **la flessibilità dei metodi all'interno della stessa classe**,
  Mentre overriding riguarda:
- **la personalizzazione del comportamento ereditato dalle classi padre**.

Introduzione ai modificatori di accesso

I **modificatori di accesso** servono a **controllare la visibilità** di classi, attributi, metodi e costruttori, stabilendo **da dove** e **da chi** possano essere utilizzati.

In Java esistono **4 livelli di accesso**:

1. `public`
2. `protected`
3. *default* (nessun modificatore)
4. `private`

I modificatori di accesso possono essere applicati a:

- **classi**
- **attributi**
- **metodi**
- **costruttori**
  Questi modificatori sono uno strumento fondamentale per applicare il **principio di incapsulamento**, già visto in precedenza:
- **l'oggetto espone solo ciò che è necessario e nasconde i dettagli interni**.

Note

Le **classi top-level** (cioè non annidate) possono essere dichiarate solo `public` oppure con accesso *default*.
Non è possibile dichiarare una classe top-level `private` o `protected`.

`public`

Un membro dichiarato `public` è **accessibile ovunque**:

- **all'interno della stessa classe**
- **da altre classi dello stesso package**
- **da classi di package diversi**

Java

```java
public class Impiegato {
    public String nome;

    public void stampaNome() {
        System.out.println(nome);
    }
}
```

tipicamente usato per:

- **classi**
- **metodi che rappresentano l'interfaccia pubblica dell'oggetto**

`private`

Un membro `private` è accessibile solo all'interno della classe in cui è dichiarato.

Java

```java
public class Impiegato {
    private double salario;

    public double getSalario() {
        return salario;
    }
}
```

È il modificatore più importante per l'**incapsulamento**:

- **impedisce accessi diretti dall'esterno**
- **obbliga a usare metodi controllati (getter e setter)**

In pratica:

> *lo stato interno dell'oggetto non deve essere manipolato direttamente.*

> **Note**
>
> Un metodo `private` **non può essere soggetto a overriding**, perché non è visibile alle sottoclassi.

Accesso *default* (nessun modificatore)

Se **non viene specificato alcun modificatore**, l'accesso è consentito:

- **all'interno della stessa classe**
- **a tutte le classi dello stesso package**

Java

```java
class Impiegato {
    int matricola;
}
```

Non è accessibile da classi di package diversi.

Questo livello di accesso favorisce la **collaborazione tra classi dello stesso package**, ma offre meno isolamento rispetto a `private`.
È molto usato **all'interno di librerie**, ma meno frequente nelle applicazioni commerciali.

`protected`

Un membro `protected` è accessibile:

- **all'interno della stessa classe**
- **alle classi dello stesso package**
- **alle sottoclassi, anche se si trovano in package diversi**

Java

```java
public class Persona {
    protected String nome;
}
```

È strettamente legato al concetto di **ereditarietà** delle classi.

> **Note**
>
> Nell'overriding **non è possibile ridurre la visibilità** di un metodo:
>
> - **ad esempio, un metodo `public` non può diventare `protected` o `private` nella sottoclasse**.

Inner Class (Classi annidate)

In Java **è possibile definire una classe all'interno di un'altra classe**.
La classe esterna viene chiamata **Top-level class**, mentre la classe interna prende il nome di **inner class**.

Perché usare le inner class?

- **Permettono di raggruppare logicamente classi correlate.**
- **Consentono di controllare la visibilità della classe interna: l'accesso è subordinato alla classe esterna e ai suoi metodi.**
- **Possono accedere direttamente ai membri privati della classe esterna, rendendo più semplice la gestione di dati strettamente correlati.**

Regole principali

1. **Non può esistere un'istanza della inner class senza un'istanza della outer class.**
2. **La dichiarazione di una inner class può usare anche i modificatori di accesso (`private`, `public`, `protected`), quindi si può rendere visibile solo all'interno della classe esterna, al package o ovunque**.

Esempio

Java

```java
public class Universita {
```

```java

    private String nome;

    public Universita(String nome) {
        this.nome = nome;
    }

    // Inner class
    public class Studente {
        private String matricola;

        public Studente(String matricola) {
            this.matricola = matricola;
        }

        public void stampaInfo() {
            // L'inner class può accedere direttamente ai membri privati della outer class
            System.out.println("Studente matricola: " + matricola + ", Università: " + nome);
        }
    }
}
```

Uso dell' inner class:

Java

```java
Universita uni = new Universita("Sapienza");
Universita.Studente s = uni.new Studente("12345");
s.stampaInfo(); // Output: Studente matricola: 12345, Università: Sapienza
```